

Fixed-Graph Vanilla and Relay Belief-Propagation Decoders in Chisel

Anonymous Authors

Abstract—This manuscript records the current design, implementation, and validation of `chipsLDPC`, a Chisel/CIRCT generator for fixed-Tanner-graph quantum low-density parity-check decoders. One shared statically wired min-sum core is controlled by two autonomous decoders: early-terminating vanilla BP and Relay-BP-S with persistent marginals, deterministic pseudorandom per-variable memory coefficients, zero-bubble leg transitions, and minimum-prior-score solution selection. Every flooding iteration uses one registered check-node clock and one registered variable-node clock. The maintained experiments specialize both decoders to the same reduced Z-check detector graph of the $[[144, 12, 12]]$ bivariate-bicycle memory circuit, compare the exact emitted SystemVerilog against an independent Scala golden model, execute deterministic shots in parallel Verilator shards, and report logical word failure and measured architecture clocks. A runtime-budget meta-experiment reuses one maximum-capacity artifact and the same shot corpus to plot failure probability against mean executed iterations. This document states the arithmetic, controller state, data-structure delta, cycle equations, simulator checks, and file-to-file workflow explicitly so that each reported quantity is traceable to an implementation symbol. FPGA synthesis, placement, frequency, resource, power, and board-execution claims remain outside the current artifact.

I. RESEARCH OBJECTIVE AND CURRENT BOUNDARY

Quantum LDPC decoding research must permit rapid algorithmic changes while keeping the generated hardware, arithmetic, and timing boundaries inspectable. The purpose of `chipsLDPC` is therefore a small collection of explicit generators rather than a general decoder framework. A parity-check matrix is elaboration-time data. Chisel constructs a fixed circuit for that graph, CIRCT emits inspectable FIRRTL-dialect MLIR and synthesizable SystemVerilog, and Verilator checks both generated designs and an exact emitted artifact.

The implemented decoder milestone contains the following:

- reusable combinational CNU and VNU kernels;
- Valls-style constant shift-add scaling and Relay-style iteration-ramp scaling as elaboration-time CNU policies;
- an FPGA-paper-style reduced Relay multiplier and deterministic per-variable LFSR coefficient source;
- an elaboration-only Tanner graph and local node-port plan;
- a full static network with one CNU per row, one VNU per column, one fixed connection per nonzero of H , and a two-phase local scheduler;
- an immediate-commit view, a registered result view, and a combinational residual/convergence checker;
- autonomous vanilla and Relay-BP controllers with optional per-frame runtime limits and backpressured outputs;

- two runnable, exact-RTL BB144 experiments sharing the same physical samples, logical observables, sharding, and reporting path;
- a runtime iteration-budget sweep over either emitted decoder; and
- a separate binary Gauss–Jordan component family for future post-processing experiments.

The shared static core instantiates the base min-sum VNU once. Vanilla BP drives its bias input from the immutable prior; Relay BP computes a new bias from the immutable prior, the registered previous marginal, and a coefficient that is stable for one leg. Thus Relay support is integrated without a second Tanner network or a duplicated VNU datapath. The emitted benchmark artifacts are single-frame engines: they do not bank independent decoder states or overlap shots. Host interfaces, FPGA implementation, and board execution remain outside the implemented boundary. The independent Gauss–Jordan and filtered OSD-0 subsystems remain available but are not instantiated in either BP-only experiment artifact described here.

II. ALGORITHMIC BACKGROUND

Let $H \in \{0, 1\}^{m \times n}$ be a parity-check matrix. Check i is adjacent to variables $\mathcal{N}(i)$ and variable j is adjacent to checks $\mathcal{M}(j)$. The incoming variable-to-check message is $R_{i,j}$, the outgoing check-to-variable message is $\sigma_{i,j}$, and S_i is a syndrome bit. The syndrome-based min-sum check update is

$$\sigma_{i,j}^{(t)} = (-1)^{S_i \oplus \bigoplus_{j' \in \mathcal{N}(i) \setminus j} [R_{i,j'}^{(t-1)} < 0]} g_t \left(\min_{j' \in \mathcal{N}(i) \setminus j} |R_{i,j'}^{(t-1)}| \right). \quad (1)$$

The variable update and marginal are

$$R_{i,j}^{(t)} = B_j^{(t)} + \sum_{i' \in \mathcal{M}(j) \setminus i} \sigma_{i',j}^{(t)}, \quad (2)$$

$$M_j^{(t)} = B_j^{(t)} + \sum_{i' \in \mathcal{M}(j)} \sigma_{i',j}^{(t)}. \quad (3)$$

The hard decision is deliberately strict:

$$\hat{e}_j^{(t)} = \text{HD}(M_j^{(t)}) = \begin{cases} 1, & M_j^{(t)} < 0, \\ 0, & M_j^{(t)} \geq 0. \end{cases} \quad (4)$$

Thus a zero marginal means no correction. The implementation takes this decision from the unsaturated working-width total and then saturates the stored marginal. Vanilla BP uses

$$B_j^{(t)} = \Lambda_j, \quad R_{i,j}^{(0)} = +\Lambda_j, \quad (5)$$

where the benchmark prior Λ_j is an unsigned quantized magnitude. For the ramp policy used by both maintained experiments, the corresponding real-valued scale factor is

$$\alpha_t = \begin{cases} 1 - 2^{-t}, & 1 \leq t \leq w_m, \\ 1, & t > w_m, \end{cases} \quad (6)$$

The exact unsigned-integer operation used in Eq. (1) is

$$g_t(x) = \begin{cases} x - \lfloor x/2^t \rfloor, & 1 \leq t \leq w_m, \\ x, & t > w_m, \end{cases} \quad (7)$$

which is also $x - (x \gg t)$ in the Chisel implementation.

Relay BP replaces the static VNU bias by a memory mixture. For Relay leg ℓ , local iteration t , coefficient $\bar{\beta}_{\ell,j} = b_{\ell,j}/2^f$, and $\gamma_{\ell,j} = 1 - \bar{\beta}_{\ell,j}$, the ideal equation is

$$B_j^{(\ell,t)} = \bar{\beta}_{\ell,j} \Lambda_j + (1 - \bar{\beta}_{\ell,j}) M_j^{(\ell,t-1)}. \quad (8)$$

The hardware deliberately uses a reduced partial-product calculation. Define x_r^+ as bit r of the unsigned magnitude $|x|$ and define the restored sign $\epsilon(x)$ by

$$|x| = \sum_{r=0}^{w_a-1} x_r^+ 2^r, \quad (9)$$

$$\epsilon(x) = \begin{cases} -1, & x < 0, \\ +1, & x \geq 0, \end{cases} \quad (10)$$

$$\rho_f(x, b) = \epsilon(x) \sum_{r=0}^{w_a-1} x_r^+ \left\lfloor \frac{b 2^r}{2^f} \right\rfloor. \quad (11)$$

The implemented Relay bias is then

$$B_j^{(\ell,t)} = \text{sat}_{w_a} (\rho_f(\Lambda_j, b_{\ell,j}) + M_j^{(\ell,t-1)} - \rho_f(M_j^{(\ell,t-1)}, b_{\ell,j})), \quad (12)$$

where every selected partial product is right-shift-truncated before the sum; this is not a full product followed by one truncation. Here

$$\text{sat}_w(x) = \min(2^{w-1} - 1, \max(-2^{w-1}, x)). \quad (13)$$

The first state after load has $M_j^{(0,0)} = \Lambda_j$. At a Relay leg boundary the newly computed saturated marginal is retained while every $R_{i,j}$ is synchronously restored to $+\Lambda_j$.

Convergence is a separate binary computation. For hard-decision vector $\hat{e}$, the residual is

$$r_i^{(t)} = S_i \oplus \bigoplus_{j: H_{i,j}=1} \hat{e}_j^{(t)}, \quad \text{converged} \iff \left(\bigvee_i r_i^{(t)} \right) = 0. \quad (14)$$

III. PROGRAMMING MODEL AND READING ORDER

A. Three distinct representations

The code uses Scala, Chisel, and emitted hardware for different purposes. A plain Scala Seq, Vector, case class, loop, or pattern match runs once while elaborating a particular circuit. A Chisel Vec, UInt, SInt, Bool, Mux, or

when describes hardware that exists at runtime. FIRRTL-dialect MLIR and SystemVerilog are generated artifacts, not additional hand-maintained decoder implementations. Keeping these levels distinct is the key to reading the repository.

For example, `nodes.checks.map` in `StaticTannerDatapath.scala` is a Scala traversal that creates one module per check during elaboration. It is not a runtime iterator. Conversely, `io.step.fire` is a hardware condition evaluated on every clock cycle. The Scala match over `CheckScale` selects hardware at elaboration time; the Chisel `MuxLookup` in the ramp policy selects a constant-shift result at runtime.

B. Module conventions

Combinational tops are `RawModules`, so they acquire no accidental clock or reset. Stateful components are `Modules` and place registers at intentional phase or protocol boundaries. `RegInit` is used for control or externally required reset state; `RegEnable` and `unreset Reg` are used for wide datapath state whose validity is guaranteed by the load/step protocol. This reset policy avoids manufacturing a large reset network for data that is overwritten before use.

Classes are `final` unless extension is part of the design. Immutable case classes carry configuration or reference values. Composition and elaboration-time policy objects are preferred to hardware inheritance. The package-restricted graph records prevent callers from constructing inconsistent local port maps while still exposing the validated result to decoder code.

C. Recommended source traversal

Table I gives the shortest path through the maintained implementation. A reader following it encounters every contract before its first important use.

IV. CONFIGURATION AND STATIC GRAPH ELABORATION

A. Validated configuration objects

Quantization stores edge magnitude width w_m and accumulator width w_a , requiring $w_a \geq w_m + 1$. `CheckConfig` adds check degree and a `CheckScale`; checks require degree at least two. `VariableConfig` accepts positive degree and derives a working width. `RelayFormat` describes an unsigned fixed-point integer coefficient b with f fractional bits and requires `betaBits > f`. The two maintained experiment profiles are deliberately distinct:

$$(w_m, w_a)_{\text{vanilla}} = (4, 7), \quad \text{scale} = \text{RampScale}(4), \quad (15)$$

$$(w_m, w_a)_{\text{Relay}} = (4, 5), \quad (w_b, f, M) = (4, 3, 8). \quad (16)$$

Here $M = 2^f = 8$ and b/M represents $\bar{\beta}$. The Python problem source uses the separate prior-quantization scale $S_\Lambda = 2$. These values are defined by `RelayDefaults.q`, `paperQ`, `paperFormat`, `priorScale`, and `memoryScale` in `Config.scala`. Keeping the seven-bit vanilla accumulator separate prevents the established vanilla experiment from changing when the five-bit Relay paper profile is selected.

TABLE I
SOURCE-ORIENTED READING ORDER.

Step	Source	Read for	Then inspect
1	Config.scala	Width, degree, scaling, and Relay defaults; all are immutable Scala values.	The require clauses, then the locked RelayDefaults.
2	graph/TannerGraph.scala	Canonical sparse H , row-major global edges, and derived column adjacency.	fromRows; note which checks happen before hardware elaboration.
3	graph/TannerNodeGraphs.scala	Translation from each global edge to local CNU and VNU ports.	connections; distinguish node ID from local port number.
4	Protocol.scala	Hardware bundles crossing arithmetic and phase boundaries.	CheckResult versus the wire-view CheckMessage.
5	Arithmetic.scala	Canonical sign, conversion, balanced sums, explicit widths, clipping, and scale policies.	Operators that expand or retain width and the elaboration/runtime split in scale.
6	TwoMin.scala and CheckNode.scala	Balanced first/second minimum, shared parity, scaling, and compressed broadcast metadata.	Why equality selects the second minimum and why duplicates remain valid.
7	VariableNode.scala	Selection, sign-magnitude conversion, one shared total, and total-minus-own messages.	VariableLogic.apply and VariableLogic.connect.
8	ConvergenceChecker.scala	Combinational $H\hat{e} \oplus S$ specialized from the Scala graph.	Native XOR and OR reductions.
9	StaticTannerDatapath.scala	Full graph elaboration, load initialization, static edge wiring, immediate commit, registered result, and message restart.	StaticVariableStage, then StaticTannerCore.
10	BpDecoder.scala	Common result bundle and the vanilla BP state machine.	VanillaBpConfig, BpDecoderResult, then VanillaBpDecoder.
11	RelayBias.scala, RelayCoefficients.scala, RelayBpDecoder.scala and	Reduced products, LFSR coefficients, Relay state, leg transitions, and solution selection.	RelayBias.reducedMultiply, then LfsrRelayCoefficients, then RelayBpDecoder.
12	Reference.scala	Independent integer equations and complete vanilla/Relay runs.	runVanilla, runRelay, and lfsrBeta.
13	BpOnlyExperiment.scala and	Packed artifact wrappers, golden records, runtime limits, and	The artifact class before the executable object.
14	RelayBpExperiment.scala	RTL emission.	
15	benchmarks/bb144/sim_main.cpp	Exact-golden and many-shot modes, sparse reconstruction, timing assertions, and logical classification.	decode, exactMain, then benchmarkMain.
16	examples/ and benchmarks/rtl_shards/	End-to-end commands, hashed shard manifests, merge rules, and reports.	Each run.sh, then the adapter, workflow, and report.
17	Generate.scala and GaussJordan/	Inspection tops and independent one-bit post-processing PEs.	Treat these as emission support and a separate subsystem.

B. One canonical graph source

The factory `TannerGraph.fromRows` is the only constructor for the static parity-check graph. Its arguments are `variableCount` and `rowOnes`. For $H \in \{0,1\}^{m \times n}$, `rowOnes(i)` lists the variable indices in $\mathcal{N}(i)$. The constructor sorts every row, rejects a nonpositive variable count, an empty check set, empty rows, duplicate or out-of-range entries, and any variable with no incident check. It then assigns edge IDs in increasing check order and increasing variable order. Column adjacency, check-edge IDs, and variable-edge IDs are derived rather than accepted as redundant inputs.

This object contains no Chisel types. It adds no ports, memories, registers, or runtime router. Graph validation and transpose construction finish before the first hardware module is created.

`TannerNodeGraphs.from` derives rooted node views without renumbering an edge. A connection record

$$(e, i, p_i, j, p_j) \quad (17)$$

contains global edgeId e , check checkId i , local checkPort p_i , variable variableId j , and local variablePort p_j . Node IDs select a generated module; port numbers select a position inside that module's degree-dependent `Vec`. Both are needed because the same global edge generally has different local positions at its two endpoints. The constructor asserts that the connection sequence still contains exactly the canonical global edge IDs.

C. Steane elaboration example

The concrete graph-level example is one CSS sector of the Steane $[[7, 1, 3]]$ code [3]. With zero-based variable indices,

$$H_{\text{Steane}} = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}. \quad (18)$$

Consequently,

$$\text{rowOnes} = [[3, 4, 5, 6], [1, 2, 5, 6], [0, 2, 4, 6]]. \quad (19)$$

The graph has three degree-four checks, seven variables with degrees $(1, 1, 2, 1, 2, 2, 3)$, and twelve edges. Its identical Hamming-code blocks $H_X = H_Z$ let the same topology represent either CSS sector independently. The exact rooted views and all twelve five-field connection records are locked by `TannerNodeGraphsSpec.scala`.

V. NUMERIC AND INTERFACE CONTRACT

Table II is normative for the current implementation.

The complete edge message `SignMag` canonicalizes zero through `Arithmetic.canonicalSign`. The CNU's `EdgeMeta.sign` is not by itself a complete sign-magnitude number: it is exclusive-parity metadata that may be one even when a subsequently selected magnitude is zero. The VNU combines the fields and canonicalizes zero during conversion. This distinction preserves a clean split between sign parity and magnitude selection.

The VNU working width is

$$w_{\text{work}} = w_a + \max(1, \lceil \log_2(d_v + 1) \rceil). \quad (20)$$

TABLE II
IMPLEMENTED FIXED-WIDTH SEMANTICS.

Property	Contract
Edge representation	Separate sign and unsigned magnitude.
Complete zero	A zero-magnitude <code>SignMag</code> is nonnegative.
CNU metadata	Sign is parity metadata until paired with the selected magnitude.
Minimum tie	Equal values occupy both minimum positions.
Selector	Use second iff the incoming magnitude equals the unscaled first minimum.
Decision tie	Marginal zero produces correction zero.
Accumulator	Signed two's-complement with an explicit guard width.
Saturation	Only at named stored or edge interfaces.
Initialization	Static V2C edges start as positive copies of their variable prior.

Every prior and incoming message is padded to this width. The balanced sum uses explicit fixed-width addition; the result is clipped once into the stored w_a marginal. Each total-minus-own result is separately converted and clipped into an unsigned w_m magnitude. In Chisel, padding, signed casts, fixed-width arithmetic, and expanding arithmetic are written explicitly, so a reader should not infer mathematical unbounded integers from a Scala-looking expression.

A. Bundle-level broadcast protocol

`Protocol.scala` makes the compression boundary visible:

- `MinPair` carries the shared first and second magnitudes;
- `EdgeMeta` carries one sign and one `useSecond` bit;
- `CheckResult` stores one `MinPair` and a vector of edge metadata, not one copied pair per edge;
- `CheckMessage` is the VNU-side wire view of one broadcast pair plus one edge's metadata; and
- `VariableResult` carries the stored marginal, hard decision, and one outgoing sign-magnitude message per incident edge.

The static graph therefore registers $2w_m$ broadcast magnitude bits per CNU, plus two metadata bits per edge. It does not register an edge-expanded array of duplicated check-to-variable magnitudes. Physical fanout and placement are left to synthesis and implementation tools; the source-level architecture does not claim a routing result before those tools run.

VI. CHECK-NODE DATAPATH

A. Canonical exclusive sign

`CheckNode` first turns every input into a canonical sign bit and forms one XOR reduction with the syndrome. The exclusive sign for port j is the global parity XOR the canonical sign of port j . This shares a parity tree across all outputs and implements Eq. (1) without building an independent XOR tree per edge.

B. Balanced duplicate-preserving two-minimum finder

`TwoMinLogic` pads a degree d_c input sequence to the next power of two P with the maximum representable magnitude. Each tournament candidate stores its winner and the direct losers encountered on that winner's path. A balanced, stable winner tree uses less-than-or-equal, so the left value wins a tie and the equal right value remains on the loser path. The first minimum is the final winner; a balanced reduction over its direct losers produces the second minimum. Thus $(2, 2, 5) \mapsto (2, 2)$ rather than dropping the duplicate.

The tournament uses $P - 1$ comparisons and the loser-path reduction uses $\log_2 P - 1$, for approximately $P + \log_2 P - 2$ comparisons. This describes the generated structure, not an achieved FPGA delay. The separate `TwoMin RawModule` is only a thin IO wrapper around the reusable logic object.

Each input whose magnitude equals the unscaled first minimum sets `useSecond`. If the first minimum is duplicated, all tied edges select the second value, which is equal and therefore still correct. No minimum index or one-hot vector is routed.

C. Scaling policies

Three Scala policy values share the same CNU:

- `NoScale` returns the minimum unchanged;
- `VallsScale(a, b)` implements $\alpha x = (x \gg a) + (x \gg b)$ with one expanding addition and unsigned clipping; and
- `RampScale(k)` implements $x - (x \gg t)$ for the elaborated choices $0 \leq t \leq k$, with an unscaled default for larger encoded controls.

Only the ramp policy elaborates the leaf `scaleShift` port; `Scala Option` and pattern matching remove it from other leaf interfaces. The two shared minima are each scaled once before broadcast, avoiding a scale unit on every edge.

The resulting CNU combinational path is conceptually

$$\text{V2C state} \rightarrow \{\text{XOR, two-min, scale}\} \rightarrow \text{CNU result register.} \quad (21)$$

VII. VARIABLE-NODE DATAPATHS

A. Shared base logic

`VariableLogic` is the common combinational implementation. Each input uses `useSecond` to select a broadcast magnitude, combines it with the edge sign, and converts the complete sign-magnitude value to a sign-extended two's-complement value. A balanced adder tree computes $M_j = \Lambda_j + \sum_i \sigma_{i,j}$ once. Each outgoing message is then

$$R_{i,j} = M_j - \sigma_{i,j}. \quad (22)$$

This total-minus-own construction replaces d_v independent adder trees with one shared sum and d_v subtractors. `VariableLogic.connect` is a small wiring helper; `VariableNode` is a combinational `RawModule` that exposes the logic through a stable bundle interface.

The base leaf accepts a signed w_a -bit prior, so leaf tests can cover positive and negative values. The current static

graph's load interface intentionally accepts an unsigned w_m -bit LLR magnitude and zero-extends it, matching the present independent-error examples with $p < 1/2$. There is no hidden hard-coded +1 prior in the generator.

B. Relay bias as a core input

Relay arithmetic is composed around the base VNU rather than implemented as a second node class. `RelayBias.reducedMultiply` in `RelayBias.scala` implements Eq. (11); `RelayBias.apply` forms Eq. (12) and performs the final five-bit signed saturation. `RelayBpDecoder` applies this function once per variable and drives the resulting vector into `core.io.bias`. Consequently `VariableLogic` and every Tanner-edge bundle are identical between vanilla and Relay BP.

The base VNU combinational path is

$$\begin{aligned} \text{CNU registers} &\rightarrow \text{select/convert} \rightarrow \text{balanced sum} \\ &\rightarrow \text{subtract/clip} \rightarrow \text{VNU state register.} \end{aligned} \quad (23)$$

VIII. CONVERGENCE CHECKER

`ConvergenceChecker` accepts the pure Scala `TannerGraph` in its constructor. Elaboration specializes one native XOR reduction per row of H . At runtime it XORs each parity with the loaded syndrome to produce the residual in Eq. (14); one OR reduction followed by negation produces `converged`. The block is a combinational `RawModule`: it owns neither state nor a stop policy. In `StaticTannerCore` it observes the combinational next VNU decisions and the registered syndrome. The core presents the checker in two forms: `commit` evaluates the combinational next VNU decisions on the terminal VNU cycle, whereas `result` presents the same information from registers on the following cycle. Autonomous controllers use `commit` to decide without adding a control bubble; the manually stepped compatibility shell exports `result`.

IX. GRAPH-GENERATED STATIC DATAPATH

A. Constructor and hierarchy

`StaticTannerCore(nodes, q, scale)` receives a fully validated `TannerNodeGraphs` plan; it does not recompute graph topology internally. The default arguments are `Quantization(4, 7)` and `RampScale(4)`. It rejects any check degree below two when elaborating the min-sum network. `StaticTannerDatapath` is a compatibility shell that ties `bias=prior`, disables message restart, and exposes the older manual load/step/result interface.

Scala map calls instantiate one `StaticCheckStage` for every check and one `StaticVariableStage` for every variable. Suggested names `cnu_i` and `vnu_j` keep the emitted hierarchy inspectable. One traversal of `nodes.connections` performs all graph wiring:

- VNU extrinsic state at `variablePort` drives CNU input at `checkPort`;
- the CNU's single registered minimum pair fans out to each adjacent VNU wire view; and

- that edge's registered sign and selector accompany the pair.

There is no runtime crossbar, adjacency memory, node-ID decoder, or graph configuration bus. A different H produces a different specialized artifact.

B. Core protocol

The shared core IO contains:

- a flipped `Decoupled` load channel carrying m syndrome bits and n unsigned priors;
- a flipped `Decoupled` step channel carrying the scale control; and
- a signed bias vector and a `restartMessages` control;
- the registered immutable prior vector;
- an immediate `commit` carrying the next n marginals and decisions, m residual bits, and `converged`; and
- a registered `result` with the same payload.

A `load.fire` event stores every syndrome and prior, sets each initial marginal to the prior, sets every initial decision to false, and initializes every variable-to-check edge to positive sign and that variable's prior magnitude. Reloading is supported when the VNU phase is not active. On an enabled VNU edge, `restartMessages=false` stores the computed extrinsic messages. If it is true, the stage still stores the newly computed marginal and decision but replaces every extrinsic message by the positive immutable prior. This selective assignment in `StaticVariableStage` is the entire datapath mechanism required for a Relay leg transition.

The top gives load priority: `step.ready` requires a prior successful load, no active VNU phase, and no asserted `load.valid`. Therefore a step cannot consume uninitialized datapath state, and an asserted load cannot silently race a step. Both result views use `Valid`, not `Decoupled`; backpressure is handled only at the enclosing autonomous decoder's frame output.

C. Two-cycle iteration

Only three control registers are reset: `loaded`, `vnuPhase`, and `resultValid`. The cycle sequence after a completed load is:

- 1) On a `step.fire` edge, every CNU stage captures its result from the current V2C state. `result.valid` remains low.
- 2) During the next cycle, `commit.valid` is high and its payload is the next VNU result computed from the CNU registers. On the closing edge, every VNU stage captures that result. The controller may also stop, advance a Relay leg, or launch sparse output from this `commit`.
- 3) `result.valid` is high in the following cycle and presents the registered marginals, decisions, residual, and convergence flag.
- 4) During that result cycle, `step.ready` is high again, so a new CNU phase may be accepted at the next edge.

The node-update latency and initiation interval are therefore two cycles per iteration. Wide syndrome, CNU-result, prior,

and VNU-state registers are not reset. Their enabling protocol guarantees they are written before use; this is intentional and is tested at the external interface. Controller-visible BP latency is therefore exactly

$$C_{BP} = 2I, \quad (24)$$

for I executed iterations; there is no extra iteration-control clock.

X. AUTONOMOUS DECODER IMPLEMENTATIONS

A. Common input and output records

`StaticDecoderInput` in `StaticTannerDatapath.scala` carries the runtime syndrome and unsigned prior vector. `BpDecoderResult` in `BpDecoder.scala` is shared by both autonomous controllers and contains success, selected correction, final marginal, reported residual, `totalIterations`, `legsExecuted`, and `solutionsFound`. Both controllers use a three-state `sIdle/sRun/sOutput` machine and a Decoupled frame output. Once in `sOutput`, the registered result remains stable until `io.out.fire`; output backpressure therefore cannot modify decoder state.

B. Vanilla BP controller

`VanillaBpConfig` stores the node plan, elaborated iteration capacity $T_{\max}$, quantization, and scale policy. Its counter width is

$$w_I = \max(1, \lceil \log_2(T_{\max} + 1) \rceil). \quad (25)$$

`VanillaBpDecoder` loads one frame, drives `core.io.bias=core.io.prior`, holds `restartMessages=false`, and asserts `step.valid` throughout `sRun`. For completed iteration t , it drives the ramp control

$$u_t = \min(t, 2^{w_u} - 1), \quad (26)$$

where w_u is `scale.controlBits`. On every `core.io.commit`, it evaluates

$$\text{stop}_{BP}(t) = \text{converged}(t) \vee [t = T_{\text{frame}}]. \quad (27)$$

The terminal commit is copied to the common result; success and `solutionsFound` are one exactly when it converged, and `legsExecuted=1`. If `runtimeIterationLimit=true`, a separate `iterationLimit` port supplies T_{frame} for each accepted frame subject to $1 \leq T_{\text{frame}} \leq T_{\max}$; otherwise the elaborated capacity is used. The implementation is in `VanillaBpDecoder` in `BpDecoder.scala`; the independent executable equation is `Reference.runVanilla` in `Reference.scala`.

C. Relay coefficient generation

`RelayCoefficientSource` in `RelayCoefficients.scala` defines a replaceable source with `frameStart`, `advanceLeg`, and one $b_{\ell,j}$ output per variable. The production `LfsrRelayCoefficients` uses one 16-bit Galois LFSR per variable. Lane j is initialized on every frame to

$$s_{0,j} = 1 + ((o + 32768j) \bmod 65535), \quad (28)$$

where o is `seedOffset`. Leg zero uses $b_{0,j} = 7$ without consuming an LFSR word. At each later-leg transition,

$$s_{\ell,j} = (s_{\ell-1,j} \gg 1) \oplus \begin{cases} 0xB400, & s_{\ell-1,j}[0] = 1, \\ 0, & s_{\ell-1,j}[0] = 0, \end{cases} \quad (29)$$

$$b_{\ell,j} = 3 + (s_{\ell,j} \& 7), \quad \ell \geq 1. \quad (30)$$

Thus later legs use $b \in [3, 10]$, or effective $\gamma = 1 - b/8 \in [-0.25, 0.625]$. Coefficients remain constant inside a leg and the complete sequence repeats across frames by design. The independent Scala implementation is `Reference.lfsrBeta`.

D. Relay-BP controller

`RelayBpConfig` in `RelayBpDecoder.scala` adds initial-leg budget T_0 , later-leg budget T_r , total hardware-leg capacity $L_{\max}$, solution target S , seed offset, and Relay format. Its maximum possible iteration count is

$$I_{\max} = T_0 + (L_{\max} - 1)T_r. \quad (31)$$

The derived counter widths are the positive bit lengths of $I_{\max}$, $\max(T_0, T_r)$, $L_{\max}$, and S . The solution-score width is the positive bit length of

$$Q_{\max} = n(2^{w_m} - 1). \quad (32)$$

`RelayBpDecoder` instantiates the same `StaticTannerCore` as vanilla BP, one coefficient source, and Eq. (12) for every variable. A leg stops on its first syndrome-consistent commit or at its local budget:

$$\text{stop}_{\ell}(t) = \text{converged}_{\ell}(t) \vee \begin{cases} [t = T_0], & \ell = 0, \\ [t = T_r], & \ell > 0. \end{cases} \quad (33)$$

Every converged leg increments the solution count, including a duplicate solution. A converged candidate is ranked by

$$Q(\hat{e}) = \sum_{j=0}^{n-1} \Lambda_j \hat{e}_j. \quad (34)$$

The candidate replaces the stored best solution only when its score is strictly smaller, so an equal-score tie retains the earliest solution. The decoder stops when the updated solution count reaches S or when no configured leg remains. Otherwise, on the terminal VNU edge, `advanceLeg` both advances the coefficients and asserts `restartMessages`. The marginal and decision from that commit are retained, V2C messages return to the immutable priors, the local iteration counter becomes zero, and Eq. (6) restarts at $t = 1$ on the next

CNU clock. Hence a leg transition consumes zero additional clocks.

On success, `correction` is the best converged candidate and the reported residual is forced to zero because that candidate was checked when it was recorded. `marginal` remains the final executed leg’s soft state, which is the intended handoff to a later post-processor. On total failure, `correction`, `marginal`, and `residual` are one coherent snapshot of the last commit. Optional `runtimeLegLimit` adds a per-frame bound $1 \leq L_{\text{frame}} \leq L_{\text{max}}$ without re-elaboration. `RelayLegTrace` emits the zero-based leg index, executed local iterations, and convergence flag on every terminal commit; it is nonblocking.

E. Data-structure delta from vanilla to Relay BP

Table III lists the additions needed to transform the vanilla controller into Relay BP. The Tanner graph, static connections, CNU/VNU payloads, arithmetic leaf, and two-clock schedule are unchanged.

XI. LEGACY 2×2 TEACHING TOP

`MinSumIteration2x2` predates the generic graph-generated top and remains as the shortest registered wiring example for

$$H = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}. \quad (35)$$

It accepts check-major input edges, instantiates two resetful `CheckNodeUnits` followed by two resetful `VariableNodeUnits`, and produces one iteration after two cycles. It has no internal iterative state or graph object. New decoder work should normally start from `StaticTannerDatapath`; the 2×2 top is retained for focused pedagogy and regression coverage.

XII. INDEPENDENT SYSTOLIC GAUSS–JORDAN SUBSYSTEM

The `GaussJordan/` subtree intentionally imports no BP protocol or decoder configuration type. It demonstrates how a later post-processing stage can remain separately elaborated, tested, emitted, and scheduled.

`GjOpcode` fixes the legacy two-bit encoding: `Pass=00`, `Swap=01`, `Add=10`, and `Lock=11`. For stored bit r and input d , `PeCol` passes d , swaps or locks by emitting r and storing d , and adds by emitting $r \oplus d$. `PeDiag` gives `reduce_sig_i` priority and emits `Swap`; otherwise a zero passes, the first one locks state, and subsequent ones emit `Add`. State, data, and opcode registers have active-high synchronous reset and hold on disabled cycles. The diagonal reduction output is combinational, reset-masked, and deliberately not enable-gated.

`TrapezoidMeshConfig` validates positive n , lifted-column count, and reduction-hop delay and derives the total column count. The elaborated mesh ties off its lower-left triangle, moves data vertically, moves opcodes horizontally, and delays reduction only between diagonal cells on enabled cycles. State exports pack A and lifted B in row-major order with coordinate zero at the least-significant bit. `FlatIO` and

explicit `desiredNames` preserve the legacy external ABI; the historical top spelling `trapeziod_mesh` is intentionally retained. A streamed row feeder, row count, and decoder/post-processor controller are not present.

XIII. EMISSION AND ARTIFACT ORGANIZATION

A. Build contract

The repository pins Mill 1.1.2, Scala 2.13.18, Chisel 7.7.0, and `ScalaTest` 3.2.19 in `build.mill`. The matching Chisel compiler plugin is enabled, and warning-oriented Scala compiler options include deprecation, feature, and initialization checks. The included Mill wrapper is the sole supported build interface. `.mill-jvm-opts` supplies the project root required by the Chisel test-directory flow. Continuous integration installs Java 17 and Verilator and runs the complete Mill test target for main-branch or tag pushes, pull requests, and manual dispatches.

B. Named generator tops

`Generate.main` maps a short research target name to a zero-argument module generator. Each invocation writes one FIRRTL-dialect MLIR file and one SystemVerilog file under a requested directory. The two artifacts come from fresh but equivalently configured elaborations of the selected closure. CIRCT emission disables random initialization, strips debug information, and enables default layer specialization so generated artifacts are deterministic and compact.

`scripts/emit.sh` keeps emission separate from tests and accepts a target and optional output directory. `scripts/verify-rtl.sh` consumes an already emitted SystemVerilog file and top name and performs standalone Verilator lint; it does not elaborate Scala or prove behavior. It disables only warning classes caused by CIRCT’s single-file naming, unused portions of intentional fixed-point intermediates, and intentionally open boundary pins. Generated directories are excluded from version control. A result-bearing emitted artifact should instead be archived immutably with graph/configuration input, Git revision, and Scala, Chisel, CIRCT, Verilator, synthesis, and FPGA tool versions.

XIV. VERIFICATION METHODOLOGY

The verification ladder crosses semantic boundaries deliberately. It does not use emitted RTL text as the golden specification.

A. Pure Scala arithmetic oracle

`Reference.scala` defines test-only messages and decoder state using ordinary `Int` and `Boolean`. Its two-minimum oracle sorts, its check and variable functions use direct integer equations, and its graph iteration follows the canonical node plan. It calls no Chisel arithmetic helper. It does share validated configuration and graph metadata with the DUT; separate graph tests therefore lock topology and port mapping before the model is used for graph-level comparisons.

`Harnesses.scala` wraps combinational `RawModules` in small clocked test tops where required by `ChiselSim`. These

TABLE III
CONCRETE ADDITIONS FROM VANILLA BP TO RELAY BP.

Concern	Vanilla representation	Relay addition and source symbol
Configuration	VanillaBpConfig: nodes, $T_{\max}$, q , scale	RelayBpConfig: $T_0, T_r, L_{\max}, S$, seed offset, RelayFormat, and derived local/leg/solution/score widths
Core inputs	bias=prior; restart false	Per-variable RelayBias(...) vector; restartMessages=advanceLeg
Controller state	iteration, frameLimit, terminal outcome	leg, frameLegLimit, localIteration, totalIterations, solutions, haveBest, bestScore, and bestCorrection
Coefficient state	None	Abstract RelayCoefficientSource; production per-VNU LFSR state and coefficient registers in LfsrRelayCoefficients
Result metadata	Common result with one-bit leg/solution counts	Same BpDecoderResult type parameterized with wider legsExecuted and solutionsFound fields
Observability	Frame result only	RelayLegTrace Valid event at each terminal VNU commit
Artifact ABI	Runtime iteration limit	Runtime total-leg limit plus leg/solution outputs and trace fields

TABLE IV
NAMED EMISSION TARGETS.

Target	Emitted top
two-min	TwoMin
check-valls	CheckNode, constant scaling
check-relay	CheckNode, ramp scaling
variable	VariableNode
iteration	legacy MinSumIteration2x2
convergence	Steane convergence checker
static-steane	full static Steane datapath
vanilla-steane	autonomous four-iteration vanilla BP
relay-steane	autonomous four-leg Relay BP
bp-filtered-osd0-steane	composed BP and filtered OSD-0
neighbour-sorter	standalone reliability sorter
pe-col, pe-diag	Gauss-Jordan leaf PEs
trapezoid-mesh	three-row example mesh

harnesses add no alternative algorithm; they expose leaf IO to the simulator.

B. Executable test inventory

Table V maps each maintained specification to its claim. This is also the recommended order for debugging a failed graph-level test.

The complete ScalaTest suite is run by `scripts/test.sh` and by continuous integration. ChiselSim exercises the generated hardware with Verilator. The standalone lint script is a separate, weaker check: it establishes that a named emitted artifact is accepted by Verilator with the selected warnings, not that its values match the algorithm.

XV. SHARED BB144 EXPERIMENTAL CONTRACT

A. Code, circuit, detector model, and samples

The two maintained decoder experiments deliberately share one problem source: `benchmarks/bb144/bb144.py`. It reconstructs the pinned gross-code $[[144, 12, 12]]$ CSS matrices from 12×6 bivariate circulants. If $X = P_{12} \otimes I_6$ and $Y = I_{12} \otimes P_6$, the implemented polynomials and parity checks are

$$A = X^3 + Y + Y^2, \quad B = Y^3 + X + X^2, \quad (36)$$

$$H_X = [A \mid B], \quad H_Z = [B^T \mid A^T], \quad (37)$$

with addition over $\text{GF}(2)$. `code_data` checks the resulting 72×144 matrices, CSS commutation, 12 logical rows, and the upstream commit recorded by `UPSTREAM_COMMIT`.

`circuit` constructs the 12-noisy-cycle memory circuit plus two zero-noise closure rounds. At physical error probability p , reset and measurement flips occur with probability p , one-qubit noisy locations use a uniform nonidentity Pauli conditioned on an event of probability p , and two-qubit noisy locations analogously use one of 15 nonidentity Paulis. `sample_physical` independently implements this circuit-level Pauli process in vectorized NumPy and returns the Z-check detector differences and 12 actual logical-observable bits. Probability point a uses deterministic seed

$$z_a = z_0 + a, \quad z_0 = 14412 \quad (38)$$

unless the command line overrides z_0 .

`z_check_model` also obtains an undecomposed Stim detector error model (DEM). A decoder column is keyed by its complete detector support and logical support. Independent effects with the same key are merged by the XOR law

$$p_{a \oplus b} = p_a(1 - p_b) + p_b(1 - p_a). \quad (39)$$

Equal detector supports with different logical supports remain distinct. For DEM-column probability $p_j \in (0, 1/2)$, `quantize` computes

$$\Lambda_j = \min \left(15, \left\lfloor 2 \ln \frac{1 - p_j}{p_j} + \frac{1}{2} \right\rfloor \right). \quad (40)$$

The min-sum CNU requires degree at least two, so `reduce_degree_one` solves every singleton row exactly. If singleton check i fixes column j then $e_j = S_i$. Let F be the fixed-column set and K the retained checks. `reduce_sample` forms

$$S'_i = S_i \oplus \bigoplus_{j \in \mathcal{N}(i) \cap F} e_j, \quad i \in K, \quad (41)$$

$$o'_q = o_q \oplus \bigoplus_{j \in \mathcal{L}(q) \cap F} e_j, \quad 0 \leq q < 12. \quad (42)$$

It then deletes the fixed variables and singleton checks and remaps column indices. The current code asserts the reduced dimensions

$$(m, n, E, k_L) = (936, 8784, 30672, 12). \quad (43)$$

For every requested p , topology must match the first point; only priors and sampled frames vary. `prepare` archives the

TABLE V
VERIFICATION INVENTORY AND IMPLEMENTED CLAIM.

Specification or executable	Claim established
ConfigSpec.scala	Relay defaults are locked; representative LLR probabilities quantize and saturate as specified.
TannerGraphSpec.scala	Rows normalize, column/check/variable views derive correctly, edge IDs are stable, and malformed adjacency is rejected.
TannerNodeGraphsSpec.scala	Exact Steane check and variable views, degrees, global-edge bijection, and all local port bindings.
TwoMinSpec.scala	Exhaustive 8^4 comparison for four three-bit inputs, including all duplicate-minimum cases.
CheckNodeSpec.scala	Exhaustive unscaled degree-three behavior over all two-bit magnitudes, signs, and syndromes; directed Valls and ramp scaling.
VariableNodeSpec.scala	Fixed-seed randomized base VNU comparison, full eight-bit hard-decision range, saturation, and registered feedback.
RelayBiasSpec.scala	Reduced partial-product truncation, signs, saturation, and five-bit paper-profile bias against an independent integer model.
RelayCoefficientsSpec.scala	Lane seeds, Galois recurrence, initial coefficient, leg stability, advancement, and repeatable frame reseeding.
ConvergenceCheckerSpec.scala	All 2^7 Steane estimates crossed with all 2^3 syndromes for $H\hat{e} \oplus S$ and convergence.
IterationSpec.scala	The legacy 2×2 result and its two-cycle validity protocol.
StaticTannerDatapathSpec.scala	Step-before-load rejection, prior-to-edge initialization, all eight Steane syndromes, four consecutive iterations, two-cycle timing, marginals, decisions, residual, and convergence against the Scala oracle.
BpDecodersSpec.scala	Vanilla and Relay frame control, runtime limits, output stalls, persistent marginals, message restart, zero-cycle leg transitions, scoring/ties, failure snapshot, production LFSRs, and external oracle fixture.
SparseBitmaskStreamerSpec.scala	Ascending sparse-one output, empty masks, banking, and backpressure stability.
BpOnlyArtifactSpec.scala	BP-only wrapper, independent model, empty/sparse corrections, exact always-ready cycle equation, and output stalls.
RelayBpArtifactSpec	
RelayBpExperiment.scala	in Default Relay-5 profile, independent Relay result, per-leg trace, sparse stream, final soft state, and exact cycle equations.
EndToEndSpec.scala	
GaussJordanPESpec.scala	
TrapezoidMeshSpec.scala	
	Strict file-schema, graph-dimension, prior-range, syndrome, and iteration validation before elaboration.
	Opcode encodings, all PE transitions, synchronous reset, disabled hold, and combinational reduction forwarding.
	Configuration rejection, boundary behavior, an independent cycle oracle under deterministic and random traffic, row-major packing, hold/reset, and reduction delays one through three.

circuit, one DEM per point, sweep.json, the complete benchmark.txt stream, and the first shot at each point as verify/p*/problem.json plus actual.json. This separation makes the static graph and runtime data auditable.

B. Artifact output and exact cycle contract

BpOnlyArtifact in BpOnlyExperiment.scala and RelayBpArtifact in RelayBpExperiment.scala expose the same packed benchmark ABI. Check zero and variable zero occupy the least-significant fields. Both wrappers include unused scope and OSD-compatible accounting ports so the common C++ transport can also serve composed decoders; for the two experiments here, osdCycles, selected, activeRows, and solverCycles are identically zero.

The dense decoder correction is snapshotted by SparseBitmaskStreamer in SparseBitmaskStreamer.scala. With bank width $W_b = 64$, correction support $\mathcal{C} = \{j : \hat{e}_j = 1\}$, correction weight $w = |\mathcal{C}|$, and occupied bank count

$$b = \left| \left\{ \left\lfloor \frac{j}{W_b} \right\rfloor : j \in \mathcal{C} \right\} \right|, \quad (44)$$

the stream emits the members of $\mathcal{C}$ in strictly increasing order. An empty correction emits no placeholder; resultValid completes the frame. With an always-ready consumer, the measured wrapper latency from the accepted input edge through result visibility is

$$C_{\text{frame}} = 2I + 1 + b + w. \quad (45)$$

The $2I$ term is Eq. (24), one clock transfers the decoder result into the streamer, each occupied bank costs one selection clock, and each set bit costs one transfer clock. If correction output is stalled for d clocks, C_{frame} increases to $2I + 1 + b + w + d$. Stalling resultReady after resultValid

holds the completed frame and does not increment the reported counter. checkSparseTiming in sim_main.cpp, BpOnlyArtifactSpec, and RelayBpArtifactSpec enforce these equations.

XVI. EXPERIMENT 1: VANILLA BP ON BB144

A. Decoder profile

The runnable entry point is examples/BivariateBicycle144BpOnly/run.sh. It elaborates VanillaBpDecoder through BpOnlyArtifact with

$$(w_m, w_a, T_{\max}) = (4, 7, 30), \quad \alpha_t \text{ given by Eq. (6).} \quad (46)$$

The decoder stops at the first converged commit or after 30 iterations. A nonconverged terminal hard decision is still streamed for diagnosis, but status is numeric 2 (bp_nonconverged) and the experiment classifies the shot as a failure. BpOnlyArtifact enables the optional runtime iterationLimit port; an emitted 30-iteration-capacity artifact can therefore execute any per-frame prefix from 1 through 30 without re-elaboration. The standalone BP-only sweep drives 30 unless the budget meta-experiment is selected.

The script defaults to 1000 deterministic shots at each $p \in \{0.001, 0.002, \dots, 0.007\}$, while P_VALUES and the second positional argument can override those choices. These defaults define a maintained executable, not a statistical sufficiency claim.

B. Traceable workflow

The BP-only workflow is the following ordered dataflow; every arrow names a materialized input or output rather than an implicit in-memory handoff.

- 1) bb144.py prepare writes source/benchmark.txt, source/sweep.json, circuit/DEM provenance, and one source/verify/p*/problem.json per probability point.

- 2) `BpOnlyExperiment.main` parses one problem with `DecoderProblem.read`, writes `artifact/config.txt` and versioned `artifact/config.json`, evaluates `BpOnlyExperiment.golden`, and emits the specialized `artifact/rtl/BpOnlyArtifact.sv` through `Generate.emitSystemVerilogFiles`.
- 3) `BpOnlySweepGolden.main` runs the independent `Reference.iterate` loop for the first shot at every p and writes `verification/p*/golden.txt`. It stops on exactly the same predicate as Eq. (27) but shares no Chisel arithmetic.
- 4) Verilator compiles the saved `SystemVerilog` together with `benchmarks/bb144/sim_main.cpp` under `BP_ONLY_ARTIFACT`. Exact mode runs every golden before any performance sweep is accepted.
- 5) `adapters/chipsldpc.py` splits the immutable benchmark stream into shot shards and records SHA-256 hashes of each input, the simulator, and `artifact/config.txt` in `parallel/jobs.json`.
- 6) `workflow.py` run executes independent Verilator processes; `workflow.py` merge revalidates hashes, row counts, probability labels, and unique shot indices before writing `raw/shots.csv`.
- 7) `report.py` validates timing sums and corpus completeness, then writes `results/results.csv`, `results/timing.csv`, `results/results.json`, and the logical-failure/cycle plots.

The emitted RTL is generated once because the graph is common across p ; priors and syndromes are runtime inputs. Comparing `source/benchmark.txt` across decoder runs proves whether the two experiments consumed an identical shot corpus.

XVII. EXPERIMENT 2: RELAY-BP-S ON BB144

A. Relay-5 parameter profile

The runnable entry point is `examples/BivariateBicycle144RelayBp/run.sh`. Its default experiment profile is

$$(w_m, w_a, f, T_0, T_r, R, S, o) = (4, 5, 3, 80, 60, 600, 5, 0), \quad (47)$$

where R is the number of noninitial legs and S is the target number of converged legs. `RelayBpExperimentProfile.decoderConfig` maps this convention to the lower-level hardware field

$$L_{\max} = R + 1 = 601, \quad I_{\max} = T_0 + RT_r = 36080. \quad (48)$$

This resolves the source paper's total-leg wording in favor of its Algorithm 1 and reference-software loop convention [2]. Setting `RELAY_R=599` instead produces exactly 600 total legs.

Leg zero uses $b = 7$, hence $\gamma_0 = 1 - 7/8 = 0.125$. Later coefficients from Eq. (30) give the effective quantized interval $\gamma \in [-0.25, 0.625]$ for the requested paper interval $[-0.24, 0.66]$. The LFSR polynomial/seed mapping is a reproducible `chipsLDPC` choice because the paper does not

publish those details; it is serialized in `artifact/config.json`. Every frame reseeds, so all shots see the same coefficient schedule and differ only in their priors and syndromes.

The script defaults to 10,000 shots at $p \in \{0.001, 0.002, 0.003, 0.004\}$; the example README shows a smaller explicit 1000-shot invocation. `P_VALUES`, positional shot count, and `RELAY_T0`, `RELAY_TR`, `RELAY_R`, `RELAY_S`, and `RELAY_SEED_OFFSET` parameterize a run. This is the Z-check detector sector of the 12-cycle memory experiment. A full XZ curve would require running and combining the independently decoded X-check sector; the current result must not be labeled as that full-code curve.

B. Workflow and Relay-specific records

The Relay script follows the same seven materialized stages as Experiment 1, with the following substitutions:

- `RelayBpExperiment.main` emits `RelayBpArtifact` and a versioned Relay configuration; `RelayBpSweepGolden.main` calls `Reference.runRelay` and `Reference.lfsrBeta`;
- Verilator compiles `sim_main.cpp` under `RELAY_BP_ARTIFACT`;
- the runtime `legLimit` selects a prefix of the emitted 601-leg capacity; and
- each terminal VNU commit adds the tuple $(\ell, I_\ell, C_\ell, c_\ell)$ to the trace, where

$$C_\ell = 2I_\ell, \quad (49)$$

$$I = \sum_{\ell=0}^{L-1} I_\ell, \quad (50)$$

$$C_{BP} = \sum_{\ell=0}^{L-1} C_\ell = 2I, \quad (51)$$

$$N_{\text{sol}} = \sum_{\ell=0}^{L-1} c_\ell. \quad (52)$$

Here $c_\ell \in \{0, 1\}$ records convergence, not logical correctness. The trace is written compactly into each raw CSV row and expanded by `report.py` into `results/relay_leg_timing.csv`, `relay_leg_timing_summary.csv`, and `relay_leg_timing.png`.

If `RELAY_REF_DIR` names the pinned editable `trmue/relay` checkout, the script first runs `benchmarks/oracles/trmue_relay.py` on the pinned repetition-code coefficient fixture and archives `verification/trmue-relay.json`. This optional cross-implementation check is separate from, and does not replace, the mandatory Scala golden and exact emitted-RTL checks.

XVIII. EXACT RTL SIMULATION AND STATISTICAL REDUCTION

A. The dual-mode `sim_main.cpp` harness

`benchmarks/bb144/sim_main.cpp` is the behavioral boundary between the saved `SystemVerilog` artifact and all

accepted experiment data. A compile-time macro selects `VBpOnlyArtifact` or `VRelayBpArtifact`; the arithmetic and controller remain entirely in emitted RTL. Template helpers `setBit`, `getBit`, `setField`, and `signedField` handle both scalar and Verilator wide ports and decode two's-complement soft fields without assuming a C++ layout for wide signals.

`readConfig` parses `artifact/config.txt` and rejects mismatched ABI versions, dimensions, widths, invalid Relay counts, or excessive capacity. For vanilla BP, a requested iteration budget B maps directly to the hardware runtime limit. For Relay BP it must satisfy

$$B = T_0 + rT_r, \quad r \in \mathbb{Z}_{\geq 0}, \quad (53)$$

and `decoderRuntimeLimit` drives

$$L_{\text{frame}} = 1 + \frac{B - T_0}{T_r}. \quad (54)$$

Both mappings are checked against the capacity recorded beside the RTL.

`decode` performs one complete hardware transaction in five explicit phases:

- 1) clear all packed inputs, set the runtime limit, and pack syndrome bit i at offset i and prior j at offset jw_m ;
- 2) assert `inputValid`, require `inputReady`, clock the accepting edge, and then deassert `inputValid`;
- 3) on each subsequent clock, collect every sparse correction transfer and Relay leg-trace event until `resultValid`;
- 4) reject any repeated or out-of-range correction index, require `correctionLast` only on the final nonempty transfer, reconstruct the dense correction, and sign-extend each packed w_a -bit soft output;
- 5) compare the RTL cycle counter with the harness's observed clock count, accept the result, and clock once more to return the wrapper to idle.

For a requested iteration bound B and configured OSD prefixes p_k , the benchmark-mode deadlock watchdog is

$$C_{\text{max}} = 4n + 2m + 3 \sum_k p_k + 2B + 128. \quad (55)$$

Exact mode substitutes the golden iteration count for B . This bound is only a failure detector; it is never reported as decoder latency.

`checkSparseTiming` enforces Eqs. (24) and (45). For these BP-only artifacts, `breakdown` consequently records

$$C_{\text{dispatch}} = 1, \quad C_{\text{BP output}} = b + w, \quad (56)$$

with all sort, solver, and OSD terms zero. `checkRelayTiming` enforces ordered leg indices, per-leg limits, Eq. (52), and consistency of the solution count. A trace or timing mismatch is therefore a failed run, not a post-processing warning.

The executable has two modes:

- `exactMain(golden, result)` reads an independent Scala golden, calls `decode` at the artifact's full configured capacity, and compares status, iteration and cycle counts, soft vector, sparse stream, and all Relay

metadata. Only after every field matches does it write `result.json`.

- `benchmarkMain(input, csv, config, budget)` reads the BB144 stream, runs every shot on the same already-verified RTL object, checks all transport/timing invariants, evaluates logical observables, and writes one raw CSV row per shot. The optional budget selects the runtime path in Eqs. (53)–(54).

The Scala golden files are produced by `BpOnlyExperiment.golden` or `RelayBpExperiment.golden`. Their configuration line, syndrome, priors, terminal status/counters, soft state, and sparse correction are a compact verification protocol; the Relay record also contains every expected leg tuple.

B. Logical classification and reported estimators

For logical-support row $\mathcal{L}(q)$, `benchmarkMain` predicts

$$\hat{o}_q = \bigoplus_{j \in \mathcal{L}(q)} \hat{e}_j. \quad (57)$$

Let a_s be the decoder-success predicate for shot s (vanilla convergence or at least one Relay solution), and let o_s be the 12-bit physical-sample observable vector after Eq. (42). The recorded word failure indicator is

$$F_s = [\neg a_s] \vee [\hat{o}_s \neq o_s]. \quad (58)$$

For N shots at one p , `report.py` computes

$$\hat{P}_{\text{word}} = \frac{1}{N} \sum_{s=1}^N F_s, \quad (59)$$

$$\hat{P}_{L/\text{cycle}} = 1 - (1 - \hat{P}_{\text{word}})^{1/12}, \quad (60)$$

$$\bar{C} = \frac{1}{N} \sum_{s=1}^N C_s. \quad (61)$$

Equation (60) converts a 12-QEC-cycle experiment word probability under a constant independent-rate interpretation; it does not change the underlying shot-level failure definition. The conditional BP plot uses only shots with successful BP status in both numerator and denominator and counts $[\hat{o}_s \neq o_s]$ among that subset.

For $x = \sum_s F_s$, $\hat{p} = x/N$, and $z = 1.959963984540054$, the reported Wilson 95% interval is

$$d = 1 + \frac{z^2}{N}, \quad (62)$$

$$c = \frac{\hat{p} + z^2/(2N)}{d}, \quad (63)$$

$$r = \frac{z}{d} \sqrt{\frac{x(N-x)}{N^3} + \frac{z^2}{4N^2}}, \quad (64)$$

$$[p_-, p_+] = [\max(0, c - r), \min(1, c + r)]. \quad (65)$$

Cycle medians, means, maxima, and nearest-rank 95th percentiles are computed from measured C_s . Stage totals are accepted only if their sum equals every shot's `total_cycles`. Relay summaries additionally aggregate the validated leg traces by probability and one-based presentation leg number.

C. Hashed parallel execution

`benchmarks/rtl_shards/adapters/chipsldpc.py` writes one input file per $(p, \text{shot range})$ and a manifest containing SHA-256 digests of every input, the simulator executable, and its configuration. With a budget sweep, the same input shard is referenced by one job per budget; no resampling occurs. `workflow.py run` skips only an already present output whose source hash, row count, local indices, and group labels all validate. It writes through a temporary file and atomically renames on success. `workflow.py merge` repeats artifact and row validation, converts local to global shot indices, and rejects duplicates before producing the raw corpus. Host wall time and throughput are recorded separately from architecture clocks.

D. Runtime iteration-budget meta-experiment

The iteration-budget example's `run.sh` is an analysis workflow over either Experiment 1 or Experiment 2, not a third decoder. It prepares one probability point and one deterministic shot corpus, emits one artifact at the maximum requested capacity, and runs every shot at every budget through the runtime-limit port. Vanilla accepts any positive budget not exceeding capacity. Relay accepts only Eq. (53); the script converts the largest budget into the emitted R capacity and the C++ harness converts each individual budget back into a total-leg limit.

For budget B and executed iterations $I_{B,s}$, `benchmarks/bb144/budget_report.py` reports

$$\bar{I}_B = \frac{1}{N} \sum_{s=1}^N I_{B,s}, \quad (66)$$

$$h_B = 1.96 \frac{s_B}{\sqrt{N}}, \quad (67)$$

$$I_{B,95} = I_{B,([0.95N])}, \quad (68)$$

$$\hat{P}_{\text{word},B} = \frac{1}{N} \sum_{s=1}^N F_{B,s}, \quad (69)$$

where s_B is the sample standard deviation and $I_{B,(k)}$ is the k th ordered value. It rejects an incomplete budget group, duplicate shot index, or any $I_{B,s} > B$. The plotted point is $(\bar{I}_B, \hat{P}_{\text{word},B})$ with the mean-iteration normal half-width and Wilson failure interval; zero observed failures are plotted at their Wilson upper bound. Outputs are `results/budget_sweep.csv`, `budget_sweep.json`, and `logical_failure_vs_average_iterations.{png,pdf}`.

E. What simulation does not establish

Behavioral tests establish finite sets of values, protocol conditions, and architecture-clock latency. A completed many-shot run estimates failure under its recorded sample distribution and reports finite-sample intervals; it does not prove the population failure rate. Neither test class establishes maximum clock frequency, critical path, lookup-table or register count,

routing congestion, memory use, power, or throughput across independently resident shots. Converting architecture clocks into time requires a named FPGA, synthesis/place-and-route version, constraints, and achieved clock period.

XIX. REPOSITORY MAP AND REPRODUCTION

The maintained repository is deliberately shallow:

- `src/main/scala/chipsldpc/` contains pure elaboration records, hardware generators, and the named emission entry point;
- `src/test/scala/chipsldpc/` contains independent arithmetic models, simulation harnesses, specifications, and golden-record generation;
- `scripts/` contains one concise action per script: test, emit, lint an emitted RTL artifact, or create the BB144 Python environment;
- `examples/` contains the maintained BB144 BP-only and Relay-BP flows plus the runtime-budget meta-experiment;
- `benchmarks/bb144/` contains their shared circuit, sampling, Verilator transport, and reporting support;
- `build.mill`, the Mill wrapper, and `.mill-jvm-opts` define the reproducible build entry; and
- `paper/main.tex` is this evolving implementation record.

IDE metadata is not part of the architecture. Generated `build/`, `test`, and TeX auxiliary products are ignored and must not be cited as immutable research outputs.

From the repository root, use one shell session for the commands below. Run the complete behavioral suite:

```
./scripts/test.sh
```

Emit and lint the static decoder as two separate steps:

```
./scripts/emit.sh static-steane
./scripts/verify-rtl.sh \
  build/generated/static-steane/\
  StaticTannerDatapath.sv \
  StaticTannerDatapath
```

Optional leaf RTL inspection uses named emission targets rather than duplicating their focused tests as examples:

```
./scripts/emit.sh pe-col
./scripts/emit.sh pe-diag
./scripts/emit.sh trapezoid-mesh
```

Run the maintained BB144 vanilla-BP benchmark with:

```
./scripts/setup-bb144.sh
cd examples/BivariateBicycle144BpOnly
OUT=../../build/generated/bb144
./run.sh ${OUT}-bp-only 1000
```

Run the Relay-BP experiment with:

```
cd ../BivariateBicycle144RelayBp
./run.sh ${OUT}-relay-bp 1000
```

The default Relay parameters are Eq. (47); the environment overrides are listed in that example’s README. Run the budget meta-experiment, selecting either `DECODER=vanilla` or `DECODER=relay`, with:

```
cd ..
cd BivariateBicycle144IterationBudgetSweep
export DECODER=relay P_VALUE=0.003
./run.sh ${OUT}-relay-budget 10000
```

Focused structure checks use Mill’s `testOnly` target with the fully qualified specification name.

When inspecting a new decoder change, the intended research loop is: change one pure configuration or leaf rule; extend the independent Scala oracle and focused specification; run that test; emit the smallest relevant top; inspect the FIRRTL-dialect MLIR for widths and structure; inspect and lint the emitted SystemVerilog; then run the exact artifact comparison if the static top is affected. This sequence localizes semantic, elaboration, lowering, and artifact-integration failures without creating parallel implementations in production source.

XX. PLANNED EXTENSIONS

The next additions should preserve the same boundaries:

- 1) synthesize and place the named BB144 artifacts, archive constraints and reports, and replace paper-clock conversions with achieved frequencies;
- 2) bank or replicate state to evaluate throughput for multiple resident syndrome streams rather than sequential single-frame latency alone;
- 3) implement and combine the complementary X-check sector under the same sample provenance before comparing against a full-code XZ curve;
- 4) compare alternative, explicitly versioned coefficient generators and, if available, reproduce the external paper’s unpublished PRNG mapping;
- 5) connect the autonomous BP result to the existing filtered OSD-0 or Gauss–Jordan subsystem under one stable artifact ABI; and
- 6) add immutable result manifests containing Git revision and exact Scala, Chisel, CIRCT, Verilator, Python, Stim, synthesis, and FPGA tool versions.

A new policy should first be a small immutable Scala choice or a narrow bundle extension. Graph topology must remain elaboration-only, and benchmarking must consume emitted artifacts rather than import the Chisel generator.

XXI. THREATS TO VALIDITY

Small exhaustive configurations expose ties and fixed-width corner cases but do not prove every degree/width combination. The Scala oracle and DUT share the validated graph metadata, although graph construction and port mapping have separate structural tests; the comparison is strong regression evidence, not a formal equivalence proof. The BB144 study covers one reduced Z-check detector sector and one circuit/noise construction. It is not a full XZ gross-code result, and exact

degree-one preprocessing changes the hardware graph while preserving the represented equations. The deterministic coefficient schedule supports reproducibility but is not evidence that the unpublished PRNG used by the external FPGA work is identical. Quantized-prior scoring is a hardware selection rule, not proof of minimum physical correction weight. Reusing the same shots across decoder/budget variants improves paired comparison but does not remove finite-sample uncertainty. Finally, Verilator clock counts are architectural; without place-and-route they imply neither frequency nor wall clock latency on an FPGA.

XXII. CONCLUSION

The present `chipsLDPC` revision implements two complete autonomous decoders over one statically wired, two-clock min-sum core. Vanilla BP adds only frame and iteration control. Relay BP extends that controller with a reduced memory-bias calculation, deterministic per-variable coefficients, leg/local/total counters, persistent marginal state, message restart, candidate scoring, and per-leg trace metadata; it does not duplicate the Tanner datapath. Both variants are emitted as exact BB144 SystemVerilog artifacts, checked against independent Scala outcomes, exercised on the same deterministic circuit-level samples, and reduced through hash-validated raw records. The explicit equations and named source symbols in this manuscript separate algorithmic behavior, architecture-clock accounting, statistical estimation, and as-yet-unmeasured FPGA implementation claims.

REFERENCES

- [1] J. Valls, F. Garcia-Herrero, N. Raveendran, and B. Vasić, “Syndrome-based min-sum versus OSD-0 decoders: FPGA implementation and analysis for quantum LDPC codes,” *IEEE Access*, vol. 9, 2021, doi: 10.1109/ACCESS.2021.3118544.
- [2] T. Maurer et al., “Real-time decoding of the gross code memory with FPGAs,” arXiv:2510.21600, 2025.
- [3] A. M. Steane, “Error correcting codes in quantum theory,” *Phys. Rev. Lett.*, vol. 77, pp. 793–797, 1996, doi: 10.1103/PhysRevLett.77.793.
- [4] M. Schoeberl, *Digital Design with Chisel*, 6th ed., 2025.
- [5] CHIPS Alliance, “Chisel documentation,” <https://www.chisel-lang.org/docs>, accessed Aug. 31, 2026.
- [6] LLVM Project, “CIRCT documentation,” <https://circt.llvm.org/docs/>, accessed Sep. 1, 2026.